

Lógica combinacional y aritmética

1st Gabriel González Muñoz

Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica

2nd Emerson Monge Hernández

Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica

Abstract—Este documento describe la unidad aritmético-lógica (ALU) y analiza cómo la frecuencia de operación de los circuitos digitales se ve limitada por factores como los tiempos de propagación, contaminación y la ruta crítica. Posteriormente, se abordan dos problemas de diseño, junto con sus soluciones propuestas y los resultados obtenidos, los cuales permitieron verificar el correcto funcionamiento de la ALU, así como evaluar sus frecuencias de operación prácticas para distintos tamaños de bits. Se confirmó que, al aumentar el tamaño, la frecuencia máxima de operación disminuye debido al alargamiento de la ruta crítica y los tiempos de propagación.

Index Terms—Unidad aritmética lógica, ruta crítica, frecuencia de operación

I. INTRODUCCIÓN

Desde su invención, la **Unidad Aritmética Lógica** (ALU) ha sido la parte fundamental de todos los computadores modernos. Es el componente fundamental que posibilita hacer cálculos de toda índole partiendo de unas pocas operaciones lógicas y aritméticas básicas.

Al momento de funcionar, la ALU recibe dos operandos y una orden de la unidad de control; luego genera el resultado y activa banderas de estado que indican condiciones específicas como resultado negativo, cero, acarreo o desbordamiento [1], [2].

Dos conceptos fundamentales son el **tiempo de propagación** (t_{pd}), definido como el retardo máximo que tarda una entrada en reflejarse en la salida, y el **tiempo de contaminación** (t_{cd}), que corresponde al retardo mínimo desde un cambio en la entrada hasta que se empieza a afectar la salida. Estos parámetros son esenciales en el análisis temporal de circuitos combinacionales y en la prevención de fallos por sincronización [3].

A raíz de los dos conceptos anteriores se tiene la **ruta crítica**, entendida como el camino más lento que atraviesa una señal en el circuito. Esta es quien finalmente determina la frecuencia máxima de operación, ya que el reloj no puede ser más rápido que el tiempo total de la ruta crítica. En arquitecturas con *pipeline*, dividir el procesamiento en etapas más cortas reduce la longitud de la ruta crítica y permite incrementar la frecuencia de operación, mejorando el rendimiento del procesador [2], [4].

Para este laboratorio se asignaron dos problemas para ser resueltos, a continuación se presentan los problemas y sus respectivas propuestas de diseño elegidas.

II. DESARROLLO

A. Problema 1

Se solicita el diseño e implementación en FPGA de una ALU parametrizable con las operaciones aritméticas de suma, resta, multiplicación, división, módulo y las operaciones lógicas and, or, xor, shift left y shift right además de las banderas de estados: Negativo (N), Cero (Z), Carry (C) y Desbordamiento (V). Para la suma, la resta y otra operación aritmética de preferencia del grupo, la implementación debe hacerse con base en tablas de verdad y circuitos básicos.

Para la solución de este problema, se utiliza el siguiente diseño para cada operación aritmética:

1) *Suma*: Para el diseño de la suma se toma como base la lógica detrás de un full adder de 1 bit el cual dispone de 3 entradas A, B, C_{in} y dos salidas Y, C_o . Las primeras dos entradas representan los sumandos y la tercera representa una entrada adicional para considerar el acarreo de una suma previa si lo hubiese. La salida Y representa el resultado de la suma y C_o el acarreo que debe tomarse en cuenta para el siguiente dígito.

El comportamiento esperado para un full adder de un bit se muestra en la siguiente tabla de verdad:

A	B	C_{in}	Y	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

TABLA I
TABLA DE VERDAD DEL FULL ADDER

2) *Resta*: Para el diseño de la resta se parte del concepto matemático de la misma, utilizando una tabla de verdad de 4 bits se ejecuta la resta de $A - B$. Sin embargo, al realizar la operación 0 - 1 naturalmente quedaría un resultado negativo, pero en el sistema binario solamente se pueden usar 0 y 1. Por lo tanto, para representar un resultado negativo se utiliza un bit de acarreo de salida (C_{out}) el cual se activa si la resta es menor a 0. Adicional a esto, se debe utilizar un bit de

acarreo de entrada (C_{in}) el cual se activa si el bit de acarreo anterior estaba activo. De esta manera se puede representar el comportamiento de la resta respetando sus respectivos acarreos y en caso de que el ultimo bit de acarreo esté activo esto activaría la flag de negativo, mostrando así que el resultado de la resta es menor que 0. Al realizar las restas con su respectivo bit de acarreo se obtiene la siguiente tabla de verdad, resumida en el diagrama de compuertas de la figura 1:

A	B	C_{in}	R	C_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

TABLA II
TABLA DE VERDAD DEL FULL SUBTRACTOR

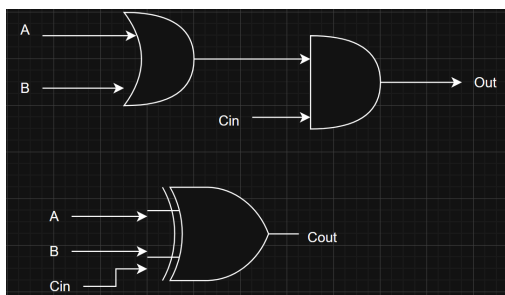


Fig. 1. Circuito de la tabla de verdad de la resta

Adicional a la suma y la resta se decide implementar de forma manual la multiplicación debido a su simplicidad en comparación a la división o el módulo, ya que con la propuesta de diseño implementada, se permite reutilizar módulos previamente creados mostrando una aplicación del modelado estructural.

3) *Multiplicación:* Para el caso de la multiplicación se parte de la definición de la misma, la cual establece que la multiplicación es la suma repetida de un mismo número. Por lo tanto se aprovecha el modulo de suma implementado previamente para usarla en cascada un numero determinado de veces, tomando el resultado de la suma anterior como el sumando A y el sumando B es el numero original. En el siguiente diagrama se muestra el funcionamiento con una cascada, este proceso se repite N veces según sea el caso.

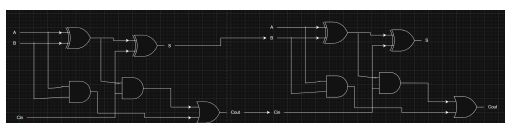


Fig. 2. Circuito representativo de la multiplicación

B. Problema 2

En sistemas combinacionales complejos, la ruta crítica es un aspecto crucial al ser un dato que determina la frecuencia de operación máxima de un sistema digital. Para este problema se solicita diseñar el circuito mostrado en la figura 3 y por medio de la herramienta TimeQuest de Altera o Timing Analyzer en el Reporte de Compilación, determinar la frecuencia de operación máxima de la ALU para 2, 4, 8, 16 y 32 bit.

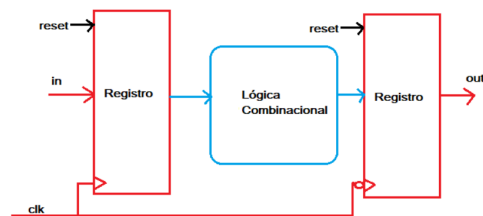


Fig. 3. Circuito para la medición de la frecuencia de operación máxima.

III. ANÁLISIS DE RESULTADOS

A. Problema 1

Para este problema el adecuado funcionamiento del diseño puede verse en el testbench creado y la simulación asociada los cuales pueden observarse en la figura 4.

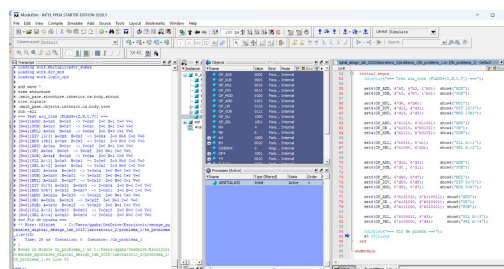


Fig. 4. Testbench del módulo de la ALU diseñada.

Se logró implementar adecuadamente el diseño en la FPGA cumpliendo con los objetivos establecidos y siguiendo los principios de modelado estructural.

B. Problema 2

En cuanto al segundo problema, con ayuda de la herramienta de Timing Analyzer se calcularon las frecuencias máximas de operación de la ALU con distintos tamaños de bits, los resultados se encuentran condensados en la tabla III y de forma gráfica en la figura 5.

Bits	Frecuencia (MHz)
2	178.19
4	99.24
8	63.21
16	31.71
32	12.77

TABLA III
FRECUENCIA DE OPERACIÓN DE LA ALU CON DISTINTOS TAMAÑOS DE BITS.

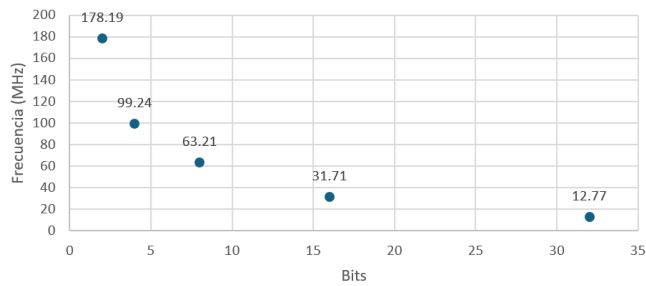


Fig. 5. Bits Vs. Frecuencia de Operación

IV. CONCLUSIONES

Existen distintas soluciones para implementar una misma operación aritmética, la elección de uno u otro está influido por los requerimientos del problema y las limitaciones del hardware. Los conceptos de tiempo de propagación, contaminación y la ruta crítica deben considerarse en cualquier diseño a fin de evitar glitches y errores de sincronización.

Para el diseño e implementación de operaciones aritméticas, el modelado estructural ofrece una ventaja al permitir el diseño de un módulo básico que posteriormente se escala para ajustarse a los requerimientos.

V. INVESTIGACIÓN

REFERENCIAS

- [1] M. M. Mano and M. D. Ciletti, *Digital Design*. Pearson, 6th ed., 2017.
- [2] D. Harris and S. Harris, *Digital Design and Computer Architecture*. Morgan Kaufmann, 3rd ed., 2021.
- [3] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*. McGraw-Hill Education, 4th ed., 2023.
- [4] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th ed., 2017.